MTL783: Theory of Computation	Date: 10/11/2025
Lecture 27	
Lecturer: Prof. Minati De	**Scribe:** Team 27

Scribed by:
1. Sudip De (2023MAS7152)
2. Aryan Sawhney (2025MAZ8291)
3. Mitali Maalav (2019MT10704)
4. Mitali Ravindra Bhagat (2020MT10821)
5. Vasundhra Singh (2024EEY7602)

Introduction and Recap

1 Space Complexity

Deterministic Turing Machine (DTM): Let M be a deterministic Turing machine that halts on all inputs. The **space complexity** of M is a function

$$f : \mathbb{N} \to \mathbb{N},$$

where $f(n)$ denotes the maximum number of tape cells that M scans on any input of length n. We then say that "M runs in space $f(n)$."

For example, if $f(n) = n^2$, then M never uses more than n^2 tape cells on inputs of length n.

Nondeterministic Turing Machine (NDTM): If M is a nondeterministic Turing machine in which all computation branches halt, its space complexity $f(n)$ is defined as the maximum number of tape cells scanned on *any branch* of computation for inputs of length n. In other words, the worst-case space among all nondeterministic paths is considered.

2 Space Complexity Classes

For a function $f : \mathbb{N} \to \mathbb{R}^+$, we define:

$$\text{SPACE}(f(n)) = \{L \mid L \text{ is decidable by a DTM using } O(f(n)) \text{ space}\}$$

$$\text{NSPACE}(f(n)) = \{L \mid L \text{ is decidable by an NDTM using } O(f(n)) \text{ space}\}.$$

Thus:

- **SPACE(f(n))** — Deterministic space complexity class
- **NSPACE(f(n))** — Nondeterministic space complexity class

3 Example (SAT in Linear Space)

Consider the NP-complete problem **SAT**. A deterministic Turing machine M_1 solving SAT can be described as follows:

M_1: On input $\langle \phi \rangle$, where ϕ is a Boolean formula:

1. For each truth assignment to the variables $x_1, \ldots, x_m$ of ϕ:
2. Evaluate ϕ on that assignment.
3. If any evaluation yields 1, accept; otherwise, reject.

Space Analysis:

- Storing a truth assignment for m variables requires $O(m)$ space.
- Each iteration reuses the same portion of the tape.
- Since $m \leq n$ (input length), M_1 runs in $O(n)$ space.

Thus, although SAT is hard in *time*, it can be solved in *linear space*, showing that space can be reused efficiently.

Example (Nondeterministic Space)

Let

$$\text{ALL}_{NFA} = \{\langle A \rangle \mid A \text{ is an NFA and } L(A) = \Sigma^*\}.$$

This is the set of all NFAs that accept every possible string.

We construct a nondeterministic machine N to decide the *complement* of this language, i.e., to find a string that the NFA rejects.

N: On input $\langle M \rangle$, where M is an NFA:

1. Place a marker on the start state of M.
2. Repeat 2^q times, where q is the number of states:
   (a) Nondeterministically select an input symbol.
   (b) Update marker positions to simulate transitions.
3. Accept if, at any point, no marker lies on an accepting state.

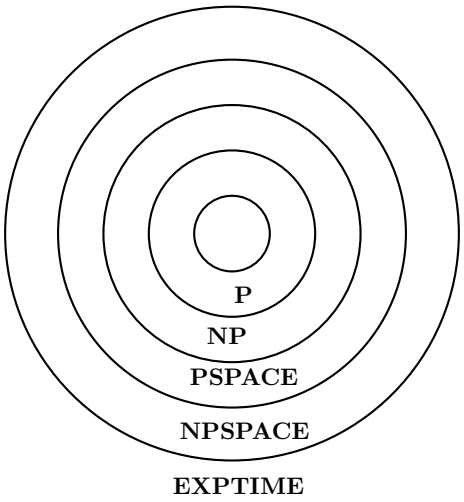
Space Analysis: The algorithm uses only linear space to store:

- Current marker positions (set of active states).
- A loop counter.

Hence, N operates in nondeterministic space $O(n)$.

Venn Diagram of Complexity Classes

$$\text{P} \subseteq \text{NP} \subseteq \text{PSPACE} \subseteq \text{NPSPACE} = \text{EXPTIME}$$



Conjecture: P $\neq$ NP

4 Theorem (Savitch's Theorem)

Savitch's Theorem

For any function $f(n) \geq log(n)$,

$$\mathbf{NSPACE}(f(n)) \subseteq \mathbf{SPACE}(f(n)^2)$$

That is, any nondeterministic computation using $f(n)$ space can be simulated deterministically using at most $f(n)^2$ space.

This result demonstrates that **nondeterminism has limited power over space**: Unlike time, where nondeterministic simulation is exponentially slower, the deterministic space overhead is only quadratic.

Proof Idea

We aim to deterministically simulate a nondeterministic TM N that operates within $f(n)$ space. A direct enumeration of all computation branches would require $2^{O(f(n))}$ space—too large. Instead, we define the *yieldability problem*:

Yieldability Problem

Given configurations c_1, c_2 of N and integer t, determine whether N can transform c_1 into c_2 within t steps using at most $f(n)$ space.

If we can solve this deterministically, then setting $c_1 = c_{\text{start}}$, $c_2 = c_{\text{accept}}$, $t = 2^{O(f(n))}$ tells us whether N accepts.

Recursive Algorithm CANYIELD

Algorithm CANYIELD

Input: c_1, c_2, t (with t a power of 2) **Output:** Accept iff N can reach c_2 from c_1 in *leqt* steps

1. If $t = 1$: Accept if $c_1 = c_2$ or c_1 yields c_2 in one step; else reject.
2. If $t > 1$: For each configuration c_m using $\leq f(n)$ space:
   (a) Run CANYIELD$(c_1, c_m, t/2)$
   (b) Run CANYIELD$(c_m, c_2, t/2)$

   Accept if both succeed; otherwise reject.

Main Simulator M

Algorithm M

1. Modify N so that upon acceptance it clears the tape and moves to the leftmost cell, entering configuration c_{accept}.
2. Let c_{start} be N's start configuration on input w.
3. Choose constant d so that N has $\leq 2^{df(n)}$ configurations.
4. Compute CANYIELD$(c_{\text{start}}, c_{\text{accept}}, 2^{df(n)})$ and output the result.

Space Analysis

Each recursive call stores only $(c_1, c_2, t) \Rightarrow O(f(n))$ space. Since each call halves t, the recursion depth is $\log t = O(f(n))$. Hence the total deterministic space is

$$O(f(n)) \times O(f(n)) = O(f(n)^2).$$

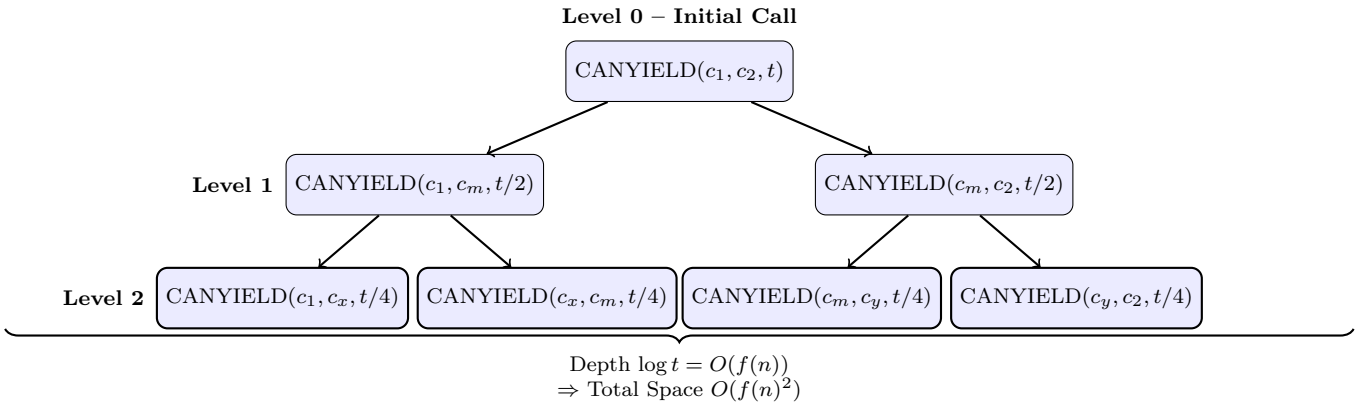
If M cannot compute $f(n)$, it can iteratively try $f(n) = 1, 2, 3, \ldots$ until convergence.

Conclusion

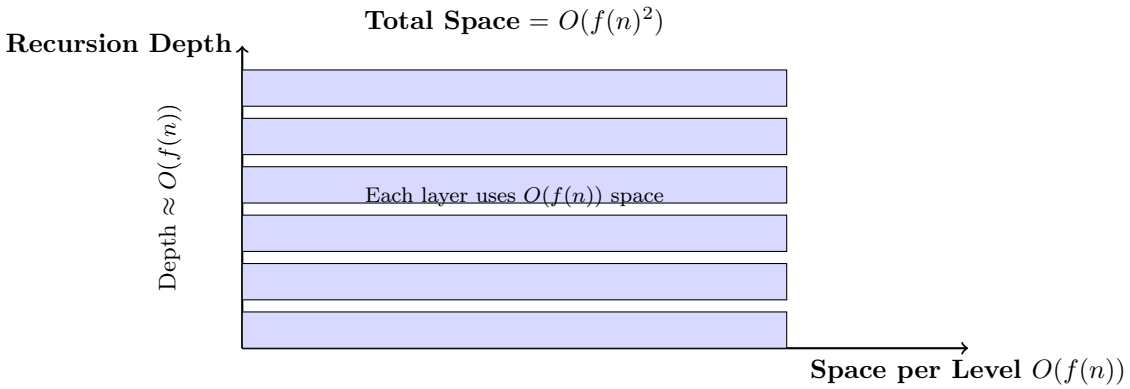
Nondeterminism increases space complexity by at most a quadratic factor:

$$\boxed{\text{NSPACE}(f(n)) \subseteq \text{SPACE}(f(n)^2)}.$$

Recursive Structure Visualization



Space Usage Visualization



References

[1] M. Sipser, *Introduction to the Theory of Computation*, 3rd Edition, Cengage Learning, 2013.